Notebook 02 - PCA Fault Detection

Model: Principal Component Analysis (PCA) -- unsupervised anomaly detection

Task: Detect whether a sample deviates from normal plant operation (binary: normal vs. fault). This is **not** multiclass classification -- the model does not identify which fault is occurring, only that something abnormal is happening. Per-fault results reported later are separate binary evaluations (one fault class at a time vs. normal), not a single model distinguishing between fault types.

Approach:

1. PCA is fitted on fault-free training data only -- it learns a compact model of what normal operation looks like
2. The number of principal components is selected at the **90% cumulative variance** threshold
3. For any new sample, two statistics are computed to measure deviation from normal:
   - **T2 (Hotelling):** Deviation *within* the PCA subspace -- captures faults that amplify normal operating patterns
   - **Q (SPE):** Reconstruction error -- captures faults that introduce patterns not seen in normal data
4. A sample is classified as faulty if **exceeds the threshold**: **Q > Q_threshold OR T2 > T2_threshold** (thresholds set at 99% confidence using empirical 99th percentile of fault-free data).
5. **Consecutive-sample rule:** An alarm is raised and a "run" is classified as faulty only when **6 consecutive samples** all exceed the threshold. Isolated crossings are discarded as noise (Russell et al., 2000)

How metrics are computed:

- **FDR, F1, Accuracy** -- computed on **raw threshold crossings** (per-sample). A sample counts as detected if it exceeds the threshold, regardless of the consecutive rule.
- **% Runs Detected, Detection delay** -- computed using the **consecutive rule** (per-run). A run counts as detected only if it produces at least one alarm (6 consecutive exceedances).
- **FPR** -- 2 variants: 1 computed using the **Sample rule** and 1 using **consecutive rule** (practical false alarm rate).

Data: Pre-scaled and pre-processed parquets from Notebook 01 (StandardScaler fitted on fault-free training data)

Fault classes: 20 fault classes (0 = normal + faults 1--20).

Section 1 - Data Loading

```
Training: 5,050,000 rows x 48 features
Test:     8,480,000 rows x 48 features
Fault classes: [np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5),
np.int64(6), np.int64(7), np.int64(8), np.int64(9), np.int64(10), np.int64(11), np.int64(12),
np.int64(13), np.int64(14), np.int64(15), np.int64(16), np.int64(17), np.int64(18), np.int64
(19), np.int64(20)]
```

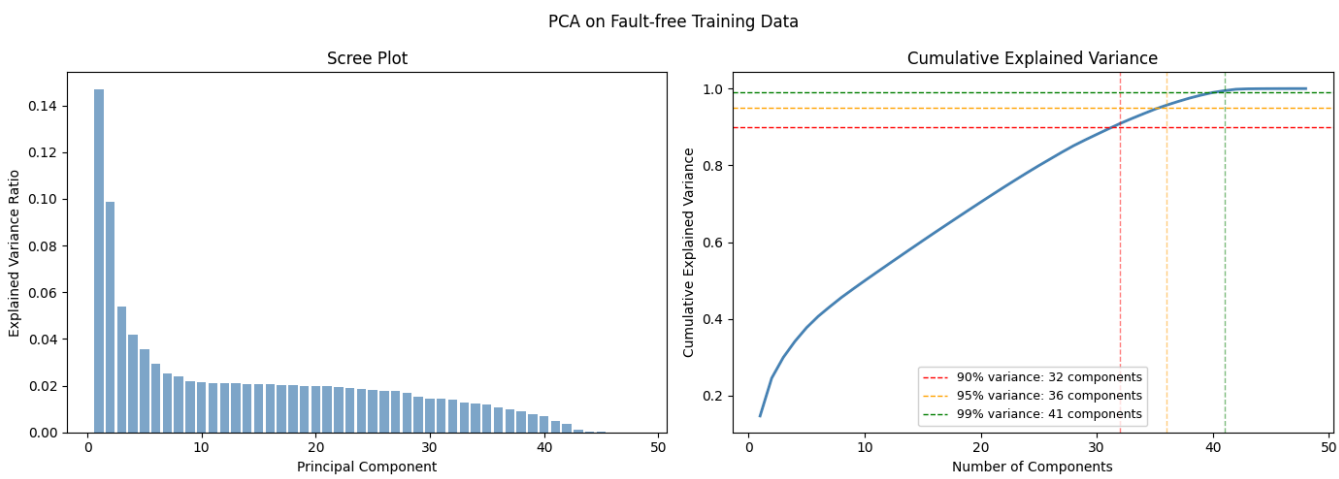
Section 2 - PCA Components

2.1 - Variance Analysis (Scree Plot)

Before choosing how many principal components to retain, we examine how variance is distributed across all components. PCA is fitted on **fault-free training data only** (already StandardScaled from Notebook 01).

- **Scree plot** (left): variance explained by each individual component -- a steep drop-off means a few components capture most of the information
- **Cumulative variance** (right): running total -- we mark the 90%, 95%, and 99% thresholds to see how many components are needed for each level

This guides the choice of `N_COMPONENTS` in the next section.




```
Components to explain 90% variance: 32
Components to explain 95% variance: 36
Components to explain 99% variance: 41
Total features: 48
Dimensionality reduction at 90%: 48 -> 32 (67% of original features)
```

Findings

- 48 features in total (after preprocessing in Notebook 01)
- 32 components explain 90% of variance in normal operation -- a reduction to 67% of the original feature count
- 36 components needed for 95%, 41 for 99% -- the curve flattens gradually with no sharp elbow, indicating variance is spread relatively evenly across many components rather than concentrated in a

few
- The scree plot confirms no single dominant component: PC1 explains ~14.5% and PC2 ~9.9%, after which individual contributions drop below 6% and continue declining smoothly

Component selection: We use the 90% variance threshold (32 components) as the cutoff for the PCA model in Section 3.

2.2 - Model Fitting

Using the variance analysis from Section 2.1, we select N_COMPONENTS at the **90% cumulative variance** threshold and fit the final PCA model on fault-free training data. The eigenvectors (P) and eigenvalues (L) are extracted for manual T2 and Q computation in the next section.

```
Components:                  32
Cumulative variance explained: 0.9088
Input feature count:         48
Fault-free training samples: 250,000
P shape: (48, 32)  (features x components)
L shape: (32,)  (one eigenvalue per component)
```

Findings

- PCA was fitted on fault-free training data only (250,000 samples -- 500 runs x 500 samples for fault class 0)
- The minimum number of components needed to explain 90% of variance was selected from the scree curve
- Input dimensionality reduced from 48 features to **32 components** (90% cumulative variance)
- Eigenvectors (P) and eigenvalues (L) are extracted directly for manual computation of T2 and Q statistics in Section 3

Section 3 - T2 and Q Statistics

Once PCA is fitted on normal data, we use it to compute two statistics for every new sample:

- **T2 (Hotelling):** Projects the sample into the PCA subspace and measures how unusual it is *along the learned normal directions*. A high T2 means the sample is moving in a direction that normal data also moves in, but by an unusually large amount.

- **Q (Squared Prediction Error / SPE):** Reconstructs the sample using the PCA model and measures how much is left over. A high Q means the sample contains variation that the PCA model of normal data cannot explain at all -- a sign that something new and unexpected is happening.

3.1 - Threshold Computation

Each statistic gets a threshold derived analytically from the fault-free training data at 99% confidence:

- T2 threshold: F-distribution (Tracy formula)
- Q threshold: Jackson-Mudholkar method (accounts for the residual eigenvalue distribution)

A sample is flagged as a fault if **Q exceeds its threshold OR T2 exceeds its threshold**. Empirical 99th percentile thresholds are used for consistency across PCA and DPCA models.

```
Analytical thresholds (99% confidence):
  T2 (F-distribution):      53.4951  (FPR: 0.0099)
  Q  (Jackson-Mudholkar):   10.5866  (FPR: 0.0090)

Empirical thresholds (99th percentile, used going forward):
  T2: 53.4467  (FPR: 0.0100)
  Q:  10.4572  (FPR: 0.0100)
```

Findings

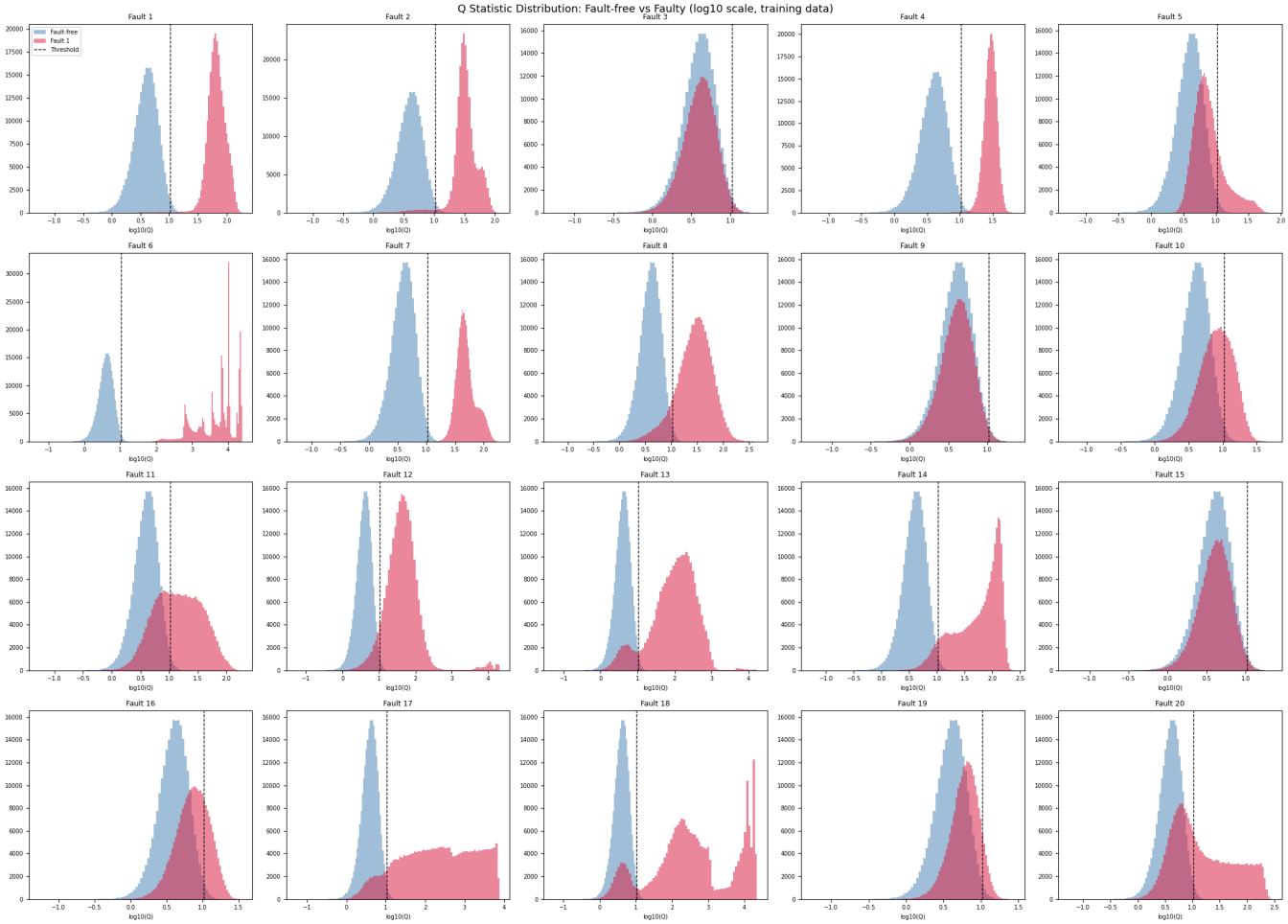
Statistic	Analytical Threshold	Empirical Threshold (99th pct)
T2	F-distribution: 53.50 (FPR 0.99%)	**53.45** (FPR 1.00%)
Q	Jackson-Mudholkar: 10.59 (FPR 0.90%)	**10.46** (FPR 1.00%)

Analytical and empirical thresholds are very close, validating the fault-free distribution assumptions. **Empirical 99th percentile thresholds** are used for both PCA and DPCA going forward to ensure consistent 1% FPR across models -- analytical thresholds can inflate FPR when independence assumptions are violated (as with DPCA lagged samples).

3.2 - Q Statistic Distributions

Histograms comparing the Q statistic (reconstruction error) for fault-free vs faulty samples across all **20 fault classes**, evaluated on the **training data**.

Clear separation between the two distributions indicates the fault is detectable by PCA. Heavy overlap indicates the fault produces patterns similar to normal operation and will be harder to detect. The empirical 99th-percentile threshold is shown as a dashed line.



Findings

Each plot shows log10(Q) for fault-free (blue) vs faulty (red), with the empirical 99th-percentile threshold as a dashed line.

- **Strong separation:** Faults 1, 2, 4, 6, 7, 14, 17 -- faulty distribution clearly beyond threshold.
- **Moderate separation:** Faults 5, 8, 11, 12, 13, 18, 20 -- shifted right but overlap near threshold; 11 and 20 appear bimodal.
- **Weak/overlapping:** Faults 3, 9, 10, 15, 16, 19 -- heavy overlap with fault-free, Q alone insufficient. Faults 3, 9, 15 are known to be undetectable by linear methods.

Section 4 - Evaluation & Analysis

4.1 - Fault Detection (by Sample and Run)

Evaluate PCA fault detection on the held-out **test set** across all **20 fault classes**.

Important: This is binary anomaly detection (normal vs. fault), not multiclass classification. Each fault class is evaluated independently -- for a given fault, we check whether the model flags those samples as anomalous compared to normal operation. The model has no knowledge of fault identity.

Detection rules (per sample, and per run):

1. For each sample, check: does **Q > Q_threshold** OR **T2 > T2_threshold**? If yes, the sample *exceeds* the threshold and is classified as faulty
2. Apply the **consecutive-sample rule**: a run triggers an **alarm** when **N_CONSEC = 6 consecutive samples** all exceed the threshold. Isolated threshold crossings are discarded as noise (Russell et al., 2000). This imposes a **minimum detection delay of 15 minutes** (5 additional samples x 3 min/sample after the first exceedance).

Sample-level metrics (computed on sample threshold crossings, no consecutive rule):

- **FDR** (Fault Detection Rate) = fraction of post-onset fault samples that exceed the threshold
- **F1** = harmonic mean of precision and recall (fault-free test data used as negatives)
- **Accuracy** = overall correct classifications
- **Global False Positive Rate FPR (sample)** = fraction of fault-free test samples that exceed the threshold

Run-level metrics (computed using the consecutive rule):

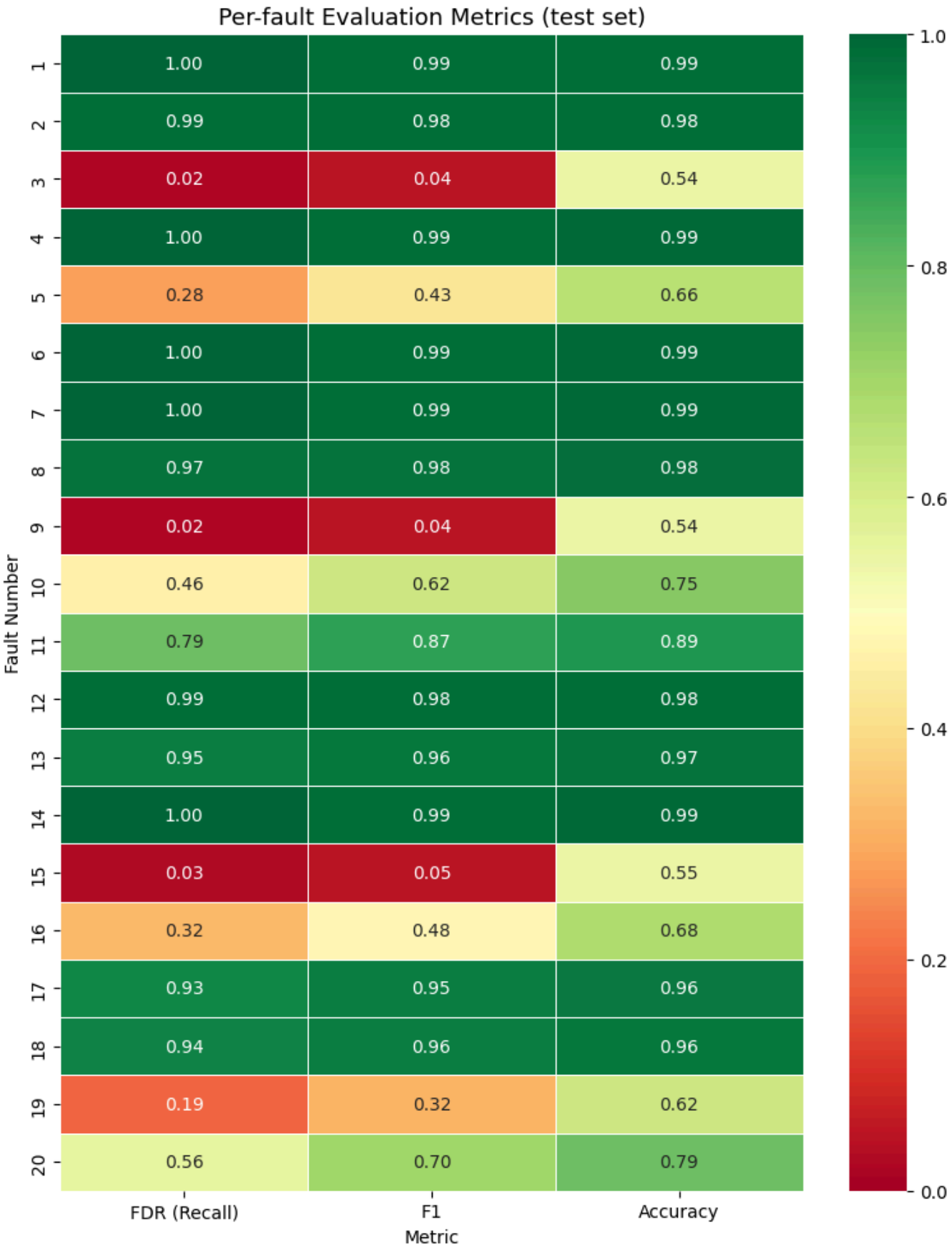
- **% Runs Detected** = fraction of test runs where at least one alarm (6 consecutive exceedances) occurred after onset
- **Global FPR (consecutive)** = fraction of fault-free test samples that are part of a 6-consecutive alarm (practical false alarm rate)
- **Detection Delay** = time from fault onset to first 6-consecutive alarm, in minutes (samples x 3 min/sample)

```
N_CONSEC = 6 (consecutive exceedances required for alarm)

       FDR (Recall)     F1  Accuracy  % Runs Detected
Fault
0               NaN    NaN    0.9794              NaN
1            0.9976 0.9866    0.9877         100.0000
2            0.9874 0.9814    0.9830         100.0000
3            0.0226 0.0432    0.5445           1.0000
4            0.9998 0.9877    0.9887         100.0000
5            0.2776 0.4263    0.6604         100.0000
6            1.0000 0.9878    0.9888         100.0000
7            1.0000 0.9878    0.9888         100.0000
8            0.9749 0.9751    0.9773         100.0000
9            0.0232 0.0444    0.5448           1.0000
10           0.4641 0.6235    0.7452         100.0000
11           0.7865 0.8684    0.8917         100.0000
12           0.9870 0.9813    0.9829         100.0000
13           0.9491 0.9617    0.9656         100.0000
14           0.9996 0.9876    0.9886         100.0000
15           0.0254 0.0484    0.5458           0.8000
16           0.3208 0.4769    0.6800         100.0000
17           0.9304 0.9518    0.9571         100.0000
18           0.9390 0.9564    0.9610         100.0000
19           0.1926 0.3164    0.6218          43.2000
20           0.5576 0.7048    0.7877         100.0000

=== Sample-level metrics (raw threshold crossings) ===
Mean FDR:          0.6718 (67.18%)
Mean F1:           0.7149
Mean Accuracy:     0.8396
Global FPR:        0.0206 (2.06%)
Overall FDR:       0.6718 (67.18%)
  Total post-onset samples: 8,000,000
  Total detected:           5,374,069
  Total missed:             2,625,931

=== Run-level metrics (consecutive rule, N_CONSEC=6) ===
Overall % Runs Detected: 82.30% (8230/10000 runs)
Global FPR:           0.000063 (0.0063%)
```


Per-fault Evaluation Metrics (test set)
Fault Number
Metric
FDR (Recall)
F1
Accuracy
1 1.00 0.99 0.99
2 0.99 0.98 0.98
3 0.02 0.04 0.54
4 1.00 0.99 0.99
5 0.28 0.43 0.66
6 1.00 0.99 0.99
7 1.00 0.99 0.99
8 0.97 0.98 0.98
9 0.02 0.04 0.54
10 0.46 0.62 0.75
11 0.79 0.87 0.89
12 0.99 0.98 0.98
13 0.95 0.96 0.97
14 1.00 0.99 0.99
15 0.03 0.05 0.55
16 0.32 0.48 0.68
17 0.93 0.95 0.96
18 0.94 0.96 0.96
19 0.19 0.32 0.62
20 0.56 0.70 0.79
1.0
0.8
0.6
0.4
0.2
0.0

```
The Confusion Matrix for Fault 1:
[[470109   9891]
 [   966 399034]]

The Confusion Matrix for Fault 2:
[[470109   9891]
 [  5044 394956]]

The Confusion Matrix for Fault 3:
[[470109   9891]
 [390960   9040]]

The Confusion Matrix for Fault 4:
[[470109   9891]
 [    91 399909]]

The Confusion Matrix for Fault 5:
[[470109   9891]
 [288949 111051]]

The Confusion Matrix for Fault 6:
[[470109   9891]
 [     0 400000]]

The Confusion Matrix for Fault 7:
[[470109   9891]
 [     0 400000]]

The Confusion Matrix for Fault 8:
[[470109   9891]
 [ 10051 389949]]

The Confusion Matrix for Fault 9:
[[470109   9891]
 [390702   9298]]

The Confusion Matrix for Fault 10:
[[470109   9891]
 [214354 185646]]

The Confusion Matrix for Fault 11:
[[470109   9891]
 [ 85417 314583]]

The Confusion Matrix for Fault 12:
[[470109   9891]
 [  5193 394807]]

The Confusion Matrix for Fault 13:
[[470109   9891]
 [ 20363 379637]]

The Confusion Matrix for Fault 14:
[[470109   9891]
 [   167 399833]]
```

```
The Confusion Matrix for Fault 15:
[[470109   9891]
 [389843  10157]]

The Confusion Matrix for Fault 16:
[[470109   9891]
 [271673 128327]]

The Confusion Matrix for Fault 17:
[[470109   9891]
 [ 27826 372174]]

The Confusion Matrix for Fault 18:
[[470109   9891]
 [ 24394 375606]]

The Confusion Matrix for Fault 19:
[[470109   9891]
 [322962  77038]]

The Confusion Matrix for Fault 20:
[[470109   9891]
 [176976 223024]]
```

Findings

Per-fault metrics (FDR = sample-level, % Runs Detected = run-level with N_CONSEC=6):

Fault	FDR (Recall)	F1	Accuracy	% Runs Detected
0 (fault-free)	--	--	97.94%	--
1	99.76%	0.9866	98.77%	100%
2	98.74%	0.9814	98.30%	100%
3	2.26%	0.0432	54.45%	1.0%
4	99.98%	0.9877	98.87%	100%
5	27.76%	0.4263	66.04%	100%
6	100.00%	0.9878	98.88%	100%
7	100.00%	0.9878	98.88%	100%
8	97.49%	0.9751	97.73%	100%
9	2.32%	0.0444	54.48%	1.0%
10	46.41%	0.6235	74.52%	100%
11	78.65%	0.8684	89.17%	100%
12	98.70%	0.9813	98.29%	100%

Fault	FDR (Recall)	F1	Accuracy	% Runs Detected
13	94.91%	0.9617	96.56%	100%
14	99.96%	0.9876	98.86%	100%
15	2.54%	0.0484	54.58%	0.8%
16	32.08%	0.4769	68.00%	100%
17	93.04%	0.9518	95.71%	100%
18	93.90%	0.9564	96.10%	100%
19	19.26%	0.3164	62.18%	43.2%
20	55.76%	0.7048	78.77%	100%

Sample-level summary:

- **Mean FDR:** 67.18% | **Mean F1:** 0.7149 | **Mean Accuracy:** 83.96%
- **Global FPR:** 2.06%

Run-level summary (N_CONSEC = 6):

- **Overall runs detected:** 82.30% (8,230 / 10,000 runs)
- **Global FPR (consecutive):** 0.0063%

By fault difficulty:

- *Strong (FDR > 90%):* Faults 1, 2, 4, 6, 7, 8, 12, 13, 14, 17, 18 -- faults 6 and 7 achieve perfect sample-level detection.
- *Moderate (FDR 40-80%):* Faults 10, 11, 20 -- all runs detected, but significant faulty samples fall below threshold.
- *Weak (FDR < 30%):* Faults 5, 16, 19 -- fault 19 worst with only 43.2% runs detected.
- *Near-zero (FDR < 3%):* Faults 3, 9, 15 -- undetectable by linear PCA.

Note: FP is global (not per-fault) since this is binary classification. Global FPR = 2.06% at sample level, dropping to 0.0063% with the consecutive rule.

Key takeaway: PCA detects 82.30% of fault runs with near-zero false alarm rate. Effective for 14/20 faults, with a clear ceiling on faults 3, 5, 9, 15, 16, 19.

4.2 - Detection Timing

How quickly does the model detect each fault after onset?

For each test run, the **detection delay** is the number of samples between fault onset (sample 160) and the first sample that is part of a **N_CONSEC consecutive alarm streak** (Q > Q_threshold OR T2 > T2_threshold for N_CONSEC samples in a row). Multiplied by 3 min/sample (TEP sampling interval) to convert to minutes.

- A delay of 0 means the alarm triggered at the very first post-onset samples
- Runs where no post-onset streak of N_CONSEC consecutive flags occurs are counted as missed and excluded from delay statistics
- **MDR** (Missed Detection Rate) = fraction of faulty runs that did **not** trigger a consecutive alarm (run-level metric, not sample-level)
- **% Runs Detected** = fraction of runs where at least one N_CONSEC-length alarm was raised after onset

```
N_CONSEC = 6 | Sampling interval: 3 min/sample | Onset: sample 160

         MDR  Mean Delay (min)  Median Delay (min)  % Runs Detected
Fault
1     0.0000           24.0000             24.0000         100.0000
2     0.0000           50.6000             51.0000         100.0000
3     0.9900         1354.2000           1368.0000           1.0000
4     0.0000           19.0000             18.0000         100.0000
5     0.0000           18.7000             18.0000         100.0000
6     0.0000           18.0000             18.0000         100.0000
7     0.0000           18.0000             18.0000         100.0000
8     0.0000           77.9000             72.0000         100.0000
9     0.9900         1585.8000           1641.0000           1.0000
10    0.0000          245.5000            201.0000         100.0000
11    0.0000           45.6000             39.0000         100.0000
12    0.0000           43.5000             39.0000         100.0000
13    0.0000          145.2000            138.0000         100.0000
14    0.0000           19.0000             18.0000         100.0000
15    0.9920         1649.2000           1909.5000           0.8000
16    0.0000          215.7000            165.0000         100.0000
17    0.0000          115.0000            108.0000         100.0000
18    0.0000          171.2000            159.0000         100.0000
19    0.5680         1110.7000           1038.0000          43.2000
20    0.0000          151.7000            147.0000         100.0000

Mean detection delay (detected runs only): 353.9 min
Median detection delay (across fault medians): 90.0 min
```

Findings

Detection delay = time from fault onset to the 6th consecutive threshold exceedance (when alarm fires). MDR = fraction of faulty runs that did not trigger a consecutive alarm. Delay computed over detected runs only.

Why N_CONSEC = 6? With independent observations and a per-sample false alarm rate of alpha = 0.01, requiring 6 consecutive exceedances reduces the false alarm probability to 0.01^6 = 10-^1^2 per sample (Russell et al., 2000). This makes spurious alarms effectively impossible while adding only a 15-minute detection lag (5 extra samples x 3 min).

- **Mean detection delay:** 353.9 min | **Median (across fault medians):** 90.0 min

By speed:

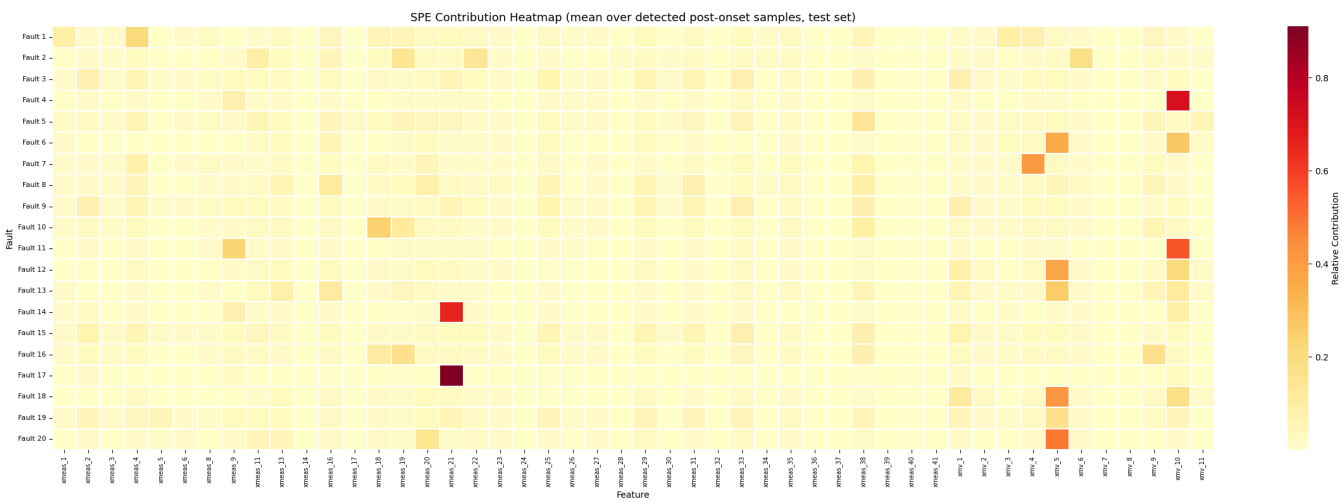
- *Immediate (median <= 24 min):* Faults 1, 4, 5, 6, 7, 14 -- alarm within minutes of onset.
- *Moderate (39-72 min):* Faults 2, 8, 11, 12 -- detected within ~1 hour.
- *Slow (> 100 min):* Faults 10, 13, 16, 17, 18, 20 -- detection takes hours due to weaker or intermittent signal.
- *Essentially undetected:* Faults 3, 9, 15 (MDR > 0.99), Fault 19 (MDR = 0.568, 43.2% runs detected).

4.3 - Contribution Heatmap

For each fault, the per-variable contribution to SPE (Q) is computed as:

```
contribution[var] = |x[var] * (x[var] - x_reconstructed[var])|
```

Averaged over all post-onset samples that were correctly detected (TP samples), this shows which features are most responsible for driving the reconstruction error during each fault. High contribution indicates the variable behaves anomalously relative to the normal PCA subspace.



Findings

The heatmap shows normalized per-variable SPE contribution averaged over detected post-onset test samples. Each row sums to 1.

Dominant features by fault:

- **xmeas_21** (Reactor Cooling Water Out) -- dominates Faults 14 and 17
- **xmv_10** (Reactor Cooling Water Valve) -- dominates Faults 4, 6, 11
- **xmv_4** (Total Feed Flow Valve) -- dominates Fault 7
- **xmv_5** (Compressor Recycle Valve) -- medium impact across Faults 6, 12, 13, 18, 19, 20

Diffuse/weak faults: Faults 3, 9, 15 -- no clear dominant feature, contributions spread thinly, consistent with near-zero detection in Section 4.1.

Overall: Most detectable faults show 1--2 dominant features. Difficult faults lack a concentrated SPE signature.

Section 5 -- DPCA Model

Static PCA treats each sample independently -- it ignores the fact that consecutive measurements in a chemical process are temporally correlated. **Dynamic PCA (DPCA)** extends PCA by augmenting each observation vector with *l* lagged observations, forming a time-lagged data matrix (Ku et al., 1995; Russell et al., 2000).

For an observation vector x_t of dimension m and lag l, the augmented vector is:

$$ ilde{x}_t = [x_t^T, x_{t-1}^T, \ldots, x_{t-l}^T]^T \in \mathbb{R}^{m(l+1)}$$

PCA is then applied to this augmented matrix. With $m = 48$ features and $l = 2$ lags, the augmented vector has $48 imes 3 = 144$ dimensions. The resulting T2 and Q statistics capture both cross-variable correlations (as in static PCA) **and** temporal autocorrelations.

Key design choice: the number of lags *l*. Yin et al. (2012) used $l = 2$ with 42 PCs (90% variance) on TEP. Russell et al. (2000) used $l = 2$-3 per fault. We follow the literature convention of $l = 2$.

5.1 -- Lag Selection & Model Fitting

1. Select lag $l = 2$ following literature convention (Yin et al., 2012; Russell et al., 2000).
2. Build the time-lagged data matrix for that lag.
3. Fit PCA on the augmented matrix, selecting components at 90% cumulative variance.
4. Compare the number of components to static PCA (expect more, since the augmented matrix has $m(l+1)$ columns).

```
Selected lag: l = 2 (literature convention)
Original fault-free shape: (250000, 48)
Lagged fault-free shape:  (249998, 144)  (m*(l+1) = 48*3 = 144)
DPCA components (90% variance): 70
Static PCA components (90% variance): 32
Cumulative variance at 70 components: 0.9033
```

Findings

- **Lag:** l = 2 (literature convention, Yin et al. 2012)
- **Augmented matrix:** 249,998 rows x 144 columns (48 features x 3 time steps)
- **DPCA components (90% variance):** 70 (vs 32 for static PCA)
- More components expected: the lagged matrix has 3x the columns, and temporal correlations introduce additional variance directions that PCA must capture.
- Yin et al. (2012) reported 42 components with l=2 on 33 variables; our 70 on 48 variables is consistent given the larger feature set.

5.2 -- Threshold Computation

Same threshold framework as static PCA (Section 3.1), applied to the DPCA model:

- T2 threshold: F-distribution at 99% confidence
- Q threshold: Jackson-Mudholkar method at 99% confidence

Thresholds are computed on fault-free training data projected through the DPCA model. Because lagged samples are temporally correlated, the Jackson-Mudholkar Q threshold can overestimate sensitivity (inflated FPR). We use **empirical 99th percentile** thresholds from the fault-free training data to guarantee 1% FPR, following the approach in Russell et al. (2000) who also adjusted thresholds to match a target significance level.

```
Analytical thresholds (99% confidence):
  T2 (F-distribution):      100.4598  (FPR: 0.0127)
  Q  (Jackson-Mudholkar):   22.7077  (FPR: 0.0555)

Empirical thresholds (99th percentile of fault-free data):
  T2: 101.9010  (FPR: 0.0100)
  Q:  28.5974  (FPR: 0.0100)

Static PCA thresholds for comparison:
  T2: 53.4467
  Q:  10.4572
```

Findings

Analytical thresholds (JM/F-distribution) gave inflated FPR for DPCA -- T2 at 1.27% and Q at 5.55% -- because lagged samples violate the independence assumption. **Empirical 99th percentile thresholds** used instead:

Statistic	DPCA Threshold	Static PCA Threshold
T2	101.90	53.45
Q	28.60	10.46

Both empirical thresholds give exactly 1% FPR on fault-free training data. DPCA thresholds are roughly 2x higher than static PCA, reflecting the larger model (70 components, 144-dim input) -- normal variation is spread across more dimensions, so the baseline T2 and Q values are higher.

Section 6 -- DPCA Evaluation & Analysis

6.1 -- Fault Detection (by Sample and Run)

Same evaluation framework as static PCA (Section 4.1). Binary anomaly detection using DPCA T2/Q statistics with empirical thresholds and N_CONSEC = 6 consecutive-sample rule. Sample-level metrics use raw threshold crossings; run-level metrics use the consecutive rule.

Note: Each test run must be converted to a lagged matrix before computing DPCA statistics. The first l samples of each run are lost to lagging.

```
N_CONSEC = 6

       FDR (Recall)     F1  Accuracy  % Runs Detected
Fault
0               NaN    NaN    0.9799              NaN
1            0.9991 0.9876    0.9886         100.0000
2            0.9889 0.9824    0.9840         100.0000
3            0.0209 0.0399    0.5446           4.2000
4            1.0000 0.9881    0.9890         100.0000
5            0.2780 0.4270    0.6613         100.0000
6            1.0000 0.9881    0.9890         100.0000
7            1.0000 0.9881    0.9890         100.0000
8            0.9759 0.9758    0.9781         100.0000
9            0.0215 0.0411    0.5448           4.0000
10           0.4128 0.5745    0.7225         100.0000
11           0.9282 0.9508    0.9564         100.0000
12           0.9901 0.9831    0.9845         100.0000
13           0.9498 0.9623    0.9662         100.0000
14           1.0000 0.9880    0.9890         100.0000
15           0.0235 0.0449    0.5458           6.2000
16           0.3279 0.4850    0.6839         100.0000
17           0.9493 0.9620    0.9660         100.0000
18           0.9417 0.9581    0.9626         100.0000
19           0.3436 0.5024    0.6911         100.0000
20           0.5745 0.7187    0.7959         100.0000

=== Sample-level metrics (raw threshold crossings) ===
Mean FDR:          0.6863 (68.63%)
Mean F1:           0.7274
Mean Accuracy:     0.8466
Global FPR:        0.0201 (2.01%)
Overall FDR:       0.6863 (68.63%)

=== Run-level metrics (consecutive rule, N_CONSEC=6) ===
Overall % Runs Detected: 85.72% (8572/10000 runs)
Global FPR (consecutive): 0.000329 (0.0329%)
```

Findings

Overall comparison (PCA vs DPCA):

Metric	PCA	DPCA	Delta
Mean FDR	67.18%	68.63%	+1.45%
Mean F1	0.7149	0.7274	+0.013
Mean Accuracy	83.96%	84.66%	+0.70%
Overall Runs Detected	82.30%	85.72%	+3.42%
Global FPR (raw)	2.06%	2.01%	-0.05%

Metric	PCA	DPCA	Delta
Global FPR (consecutive)	0.0063%	0.0329%	+0.027%

Per-fault highlights (FDR):

Fault	PCA	DPCA	Change	Note
11	78.65%	92.82%	+14.2%	Biggest gain -- temporal dynamics help
19	19.26%	34.36%	+15.1%	Nearly doubled, still weak
17	93.04%	94.93%	+1.9%	Slight improvement
10	46.41%	41.28%	-5.1%	Slightly worse
3	2.26%	2.09%	~same	Still undetectable
9	2.32%	2.15%	~same	Still undetectable
15	2.54%	2.35%	~same	Still undetectable

Run-level: faults 3, 9, 15 improved slightly (PCA: ~1% -> DPCA: 4-6% runs detected) but remain essentially undetectable.

Takeaway: DPCA gives modest overall improvement (+3.4% runs detected), with the biggest gains on Fault 11 and Fault 19 -- both faults where temporal autocorrelation carries discriminative signal. Faults 3, 9, 15 remain a hard ceiling for linear methods. Consistent with Russell et al. (2000): DPCA gives "similar performance as PCA for most faults."

6.2 -- Detection Timing

Same framework as PCA (Section 4.2). Detection delay = time from fault onset to the 6th consecutive threshold exceedance. MDR = fraction of faulty runs that did not trigger a consecutive alarm.

```
N_CONSEC = 6 | Sampling interval: 3 min/sample | Onset: sample 160

         MDR  Mean Delay (min)  Median Delay (min)  % Runs Detected
Fault
1     0.0000           26.2000             27.0000         100.0000
2     0.0000           51.9000             51.0000         100.0000
3     0.9580         1022.3000           1038.0000           4.2000
4     0.0000           24.0000             24.0000         100.0000
5     0.0000           24.0000             24.0000         100.0000
6     0.0000           24.0000             24.0000         100.0000
7     0.0000           24.0000             24.0000         100.0000
8     0.0000           80.5000             75.0000         100.0000
9     0.9600         1004.1000           1084.5000           4.0000
10    0.0000          229.0000            183.0000         100.0000
11    0.0000           37.5000             33.0000         100.0000
12    0.0000           42.3000             39.0000         100.0000
13    0.0000          147.2000            138.0000         100.0000
14    0.0000           24.1000             24.0000         100.0000
15    0.9380          981.4000            816.0000           6.2000
16    0.0000          164.4000            135.0000         100.0000
17    0.0000          115.3000            108.0000         100.0000
18    0.0000          166.9000            153.0000         100.0000
19    0.0000          259.7000            184.5000         100.0000
20    0.0000          151.8000            150.0000         100.0000

Mean detection delay (detected runs only): 230.0 min
Median detection delay (across fault medians): 91.5 min
```

Findings

Detection timing comparison (PCA vs DPCA):

Metric	PCA	DPCA	Delta
Mean delay	353.9 min	230.0 min	-123.9 min
Median delay (across faults)	90.0 min	91.5 min	+1.5 min

Per-fault highlights:

Fault	PCA Median (min)	DPCA Median (min)	PCA % Runs	DPCA % Runs	Note
19	1038.0	184.5	43.2%	**100%**	Dramatic improvement -- previously worst fault
11	39.0	33.0	100%	100%	Faster detection
16	165.0	135.0	100%	100%	-30 min
10	201.0	183.0	100%	100%	-18 min
3	1368.0	1038.0	1.0%	4.2%	Still near-zero
9	1641.0	1084.5	1.0%	4.0%	Still near-zero

Fault	PCA Median (min)	DPCA Median (min)	PCA % Runs	DPCA % Runs	Note
15	1909.5	816.0	0.8%	6.2%	Still near-zero

Key findings:

- **Fault 19:** The biggest win -- DPCA detects 100% of runs (vs 43.2% for PCA) with median delay of 184.5 min (vs 1038.0 min). Temporal dynamics are critical for this fault.
- **Mean delay dropped** from 353.9 to 230.0 min, driven largely by Fault 19 no longer being mostly missed.
- **Faults 3, 9, 15** improved slightly (1-6% runs detected) but remain essentially undetectable by linear methods.
- **Easy faults** (1, 4, 5, 6, 7, 14) have slightly higher minimum delay (24 min vs 18 min for PCA) due to losing the first 2 samples to lagging.

6.3 -- PCA vs DPCA Summary

Metric	PCA	DPCA
Components (90% variance)	32	70
Mean FDR	67.18%	68.63%
Mean F1	0.7149	0.7274
Overall Runs Detected	82.30%	85.72%
Mean Detection Delay	353.9 min	230.0 min
Median Detection Delay	90.0 min	91.5 min
Global FPR (consecutive)	0.0063%	0.0329%

What DPCA improved:

- Fault 19: 43.2% -> 100% runs detected (biggest win)
- Fault 11: FDR 78.65% -> 92.82%, detection 6 min faster
- Faults 3, 9, 15: slight improvement in runs detected (~1% -> 4-6%) but still near-zero

What didn't change much:

- Strong faults (1, 2, 4, 6, 7, 8, 12, 13, 14, 17, 18) -- already near-perfect under PCA
- Faults 3, 9, 15 -- remain undetectable by any linear method

Trade-offs:

- 2x more components (70 vs 32) and higher computational cost (per-run lagging)
- Slightly higher FPR (0.033% vs 0.006%), still negligible
- Easy faults detect ~6 min slower due to samples lost to lagging

Conclusion: DPCA is a worthwhile extension for TEP -- modest sample-level gains but a meaningful run-level improvement, especially on Fault 19. However, faults 3, 9, 15 remain a hard ceiling for linear PCA-family methods, motivating nonlinear or supervised approaches.